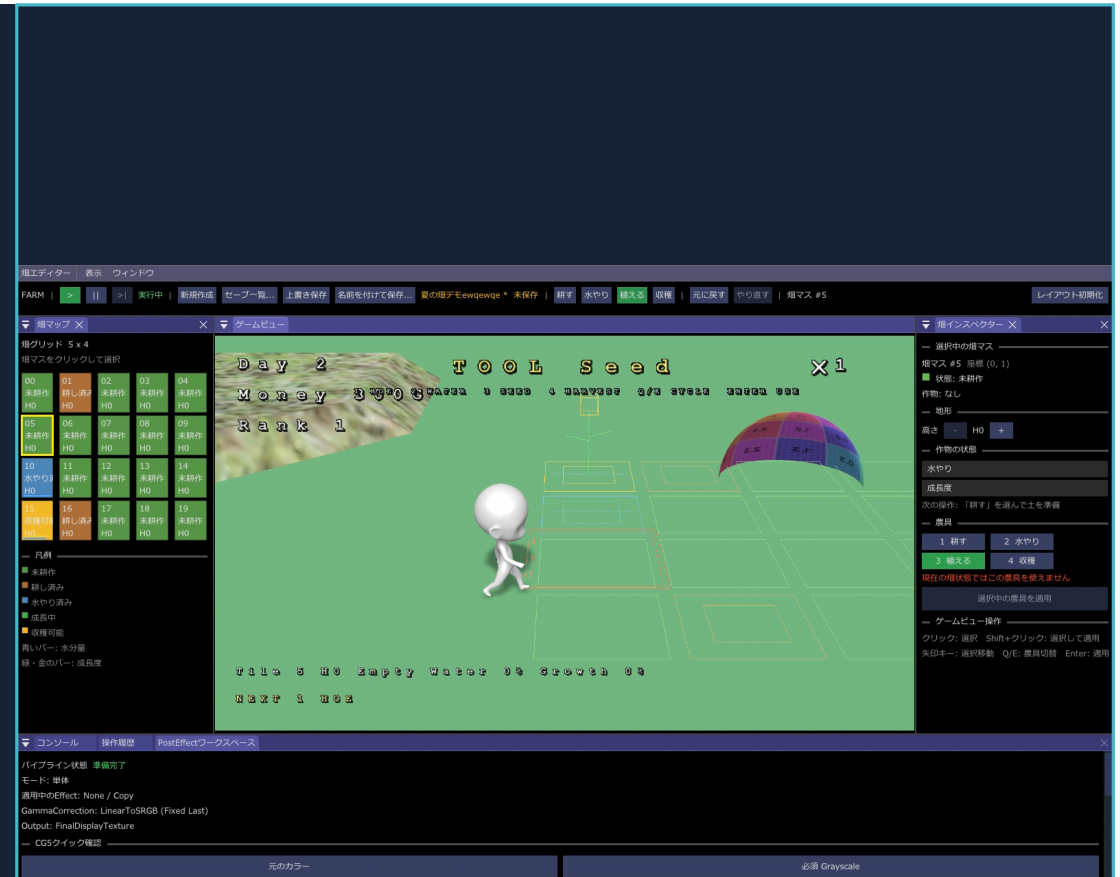


農業ゲーム（制作中）

農業ゲーム機能の責務分離、状態遷移、実行時安全性、
DirectX 12基盤との接続をコード構成に沿って説明します。

個人制作 / C++20・DirectX 12
ヨシノ ゲント
LE3C_26_ヨシノ_ゲント
2026年8月1日



01 対象・ビルド情報

項目	内容
対象	個人制作の3D農業ゲーム（制作中）
言語 / API	C++20 / DirectX 12 / Win32
IDE / Toolset	Visual Studio / Platform Toolset v145 / Windows SDK 10.0
外部ライブラリ	Dear ImGui、Assimp、DirectXTex、nlohmann/json
成果物	Release x64 executable、HLSL compiled shader objects、Resources、source code
当日検証	CG2_00_01.sln の Release x64 Rebuild成功、生成exeを作業ディレクトリprojectで10秒間起動継続

資料の範囲

本資料は農場機能と、それを支えるゲーム基盤の接続を中心に説明します。既存の描画デモやパーティクル機能を農業ゲーム独自の成果として水増ししません。

実行前提

Release版は実行ファイルと同じ階層にResources、Settings、dxcompiler.dll、dxil.dll、各.csoを配置します。相対パスでアセットを読むため、同梱フォルダ内のexeを直接起動してください。

注意

GPU・ドライバ環境差を完全には検証できていないため、別PCでの最終起動確認は残課題です。

02 責務分担

層 / クラス	担当責務	禁止している責務
Game / Framework	起動、メインループ、各Managerの寿命	農場ルールやUI判断
GamePlayScene	Systemの初期化、処理順、ViewDataの組立	個々の農作業ルールの直接実装
FarmGrid / FarmTile	畑データ、選択位置、Snapshot	入力、描画、保存I/O
Farm*System	入力解釈、操作判定、成長、日付、保存、可視化	Scene遷移、UIレイアウト
FarmHUD	ViewDataから文字列・Spriteを更新して描画	FarmGridの変更や保存処理
Engine rendering	DX12 resource、descriptor、barrier、drawの管理	農業ゲームのルール

依存方向：入力とUIは要求・表示だけを担当し、状態変更はSystemを経由します。FarmVisualSystemはFarmGridを読み取り専用として扱い、レンダリングとピッキングで同じレイアウト情報を共有します。

03 1フレームの処理順

順序	処理	理由
1	real delta time / input request取得	入力をフレーム冒頭で一度だけ収集
2	FarmInputSystem	畑選択・道具変更・操作要求を先に確定
3	camera / player input	畑が矢印入力を消費した場合は移動と競合させない
4	FixedUpdate: FarmGrowthSystem / FarmDateSystem	ゲーム状態を固定ステップで更新
5	Object3d / Lighting / Particle update	GPUへ渡す1フレーム分の状態を構築
6	shadow pass → object pass → particles	shadow write完了後にcolor passから参照
7	FarmHUD draw	ゲーム状態を変更しない最終overlay

Fixed-step

成長、日付、プレイヤー移動はFixedUpdateへ配置。可変フレーム時間による挙動差を抑えます。

入力競合の回避

ImGuiのkeyboard capture、camera drag、viewport focusを確認してから農場入力を処理します。

描画順

Directional shadow passを先に完了し、color passとparticle drawを経てHUDを最後に重ねます。

04 農場データと状態遷移

操作	事前条件	更新後
HOE	Empty	Tilled
WATER	Tilled または Planted、moisture < 1	moisture = 1
SEED	Tilled かつ crop == None	Planted / TestCrop / growth = 0
GROWTH	Planted かつ未成熟	moisture依存でgrowth増加、moisture減少
HARVEST	Planted かつ cropあり、growth >= 1	Tilled / cropなし / growth = 0

FarmTile

heightLevel / state / crop / moisture / growth の最小データで1区画を表現します。GetTile系は範囲外でnullptrを返し、変更はSetTileまたはSystem経由です。

成長式

乾燥時は25秒で100%、最大水分時は10秒で100%となる線形補間です。水分は60秒で1.0減少し、すべて0~1へclampします。NaN / Inf / 非正のdelta timeは無視します。

高さ編集

PageUp / PageDownで0~2の範囲を変更。FarmToolActionSystemのconstexprへ範囲を集約しています。

05 操作の安全性とUndo / Redo

実行前評価

EvaluateToolは対象indexと現在状態を検証し、Applied / Harvested / InvalidTarget / InvalidState / AlreadyWatered / NotReady / UnsupportedToolを返します。成功しない操作はデータを変更しません。

Command化

変更前後のFarmTileと対象indexをFarmTileEditCommandへ保持し、CommandHistoryへunique_ptrで所有権を移します。履歴上限は128件です。

寿命の前提

CommandはScene所有のFarmGridを非所有参照します。GamePlaySceneのメンバー破棄順で履歴がFarmGridより先に破棄される前提をコードコメントで明示しています。

範囲外アクセス

tileIndexを取得するたびにGetTile / GetMutableTileのnullptrを確認。viewport rayもNDC範囲・有限値・方向長を検証します。

Redo分岐

Undo後に新しいCommandを実行した場合、cursorより後ろのRedo履歴を破棄して履歴を一本化します。

不要な変更を記録しない

beforeとafterが同一ならInvalidStateとして扱い、Commandの確保と履歴追加を行いません。

06 Save / Loadの破損対策

検証対象	対策
JSON構造	必須keyと型を確認。読込後にFarmGrid::Snapshotへ変換
grid寸法	正のwidth / height、tile数と積の一致を確認
enum	FarmTileState / CropTypeの許可値だけを受理
数値	moisture / growthのfiniteと0～1、heightLevelの0～2を確認
状態整合性	Emptyにcropやgrowthを持たせない等、組み合わせを検証
保存	temporary fileへ書出し後、既存ファイルと置換
文書管理	ID・表示名・保存日時をcatalogに保持。Save As / Rename / Delete対応

UIとの境界

FarmControllerWindow等はボタン入力をFarmDocumentSystemへ通知するだけで、filesystemやJSONを直接扱いません。状態変更が成功したときだけdirty状態を更新します。

復元時の扱い

検証済みSnapshotだけをFarmGridへ適用し、Undo/Redo履歴をclearします。保存データと古い履歴の混在を防ぎます。

残る課題

I/Oエラーの自動試験、異常終了時のtemp回収、schema version移行テストは未整備です。

07 DirectX 12基盤との接続

確認観点	現在の方針 / 注意点
Resource ownership	Object3dCommon、TextureManager、ParticleManager等にGPU resource管理を集約
Descriptor	SRV割当はSrvManager経由。農業Systemはdescriptorを直接操作しない
Barrier	Shadow pass / color pass、compute particle / graphics SRV readの境界を描画基盤で管理
Fence / lifetime	Framework終了順とResourceLeakCheckerで寿命を監視。農場層はCOM resourceを所有しない
DrawCall	畑はLineDrawerへまとめて要求。ただし作物表示増加時のbatch化は今後の課題
CPU cost	成長更新は全tile走査。現在20tileでは軽量だが、大規模化時はdirty/list方式を検討
GPU cost	shadow、post effect、particleを併用。農場MVPでは不要機能を切れる構成が望ましい

農場ルール層をDX12から独立させたことで、状態遷移のテストや将来の描画方式変更を行いやすくしています。一方、Sceneには既存デモ機能も残っており、採用作品としては描画featureの登録・実行順をさらに分離する余地があります。

08 主要ソースとビルド手順

パス	役割
application/farm/core/FarmGrid.*	grid、selection、Snapshot、範囲検証
application/farm/system/FarmToolActionSystem.*	操作条件、状態変更、Command生成
application/farm/system/FarmGrowthSystem.*	水分依存の成長
application/farm/system/FarmDocumentSystem.*	保存・読込・catalog・検証
application/farm/system/FarmVisualSystem.*	畑描画とray picking
application/farm/ui/FarmHUD.*	読み取り専用ViewDataの表示
application/scene/GamePlayScene.*	各Systemの初期化と処理順
engine/command/CommandHistory.*	汎用Undo / Redo履歴

ビルド

1. Visual Studioで CG2_00_01.sln を開く
2. Configuration: Release / Platform: x64
3. Rebuild Solution
4. 生成先: ../generated/outputs/Release

実行

同梱の 03_農業ゲーム作品/01_実行ファイル/CG2_00_01.exe を起動します。Resources等を移動しないでください。

主な操作

矢印: 畑選択 / 1~4: 道具 / Q・E: 切替 / Enter: 使用

T: 時間倍率 / Y: 1日進める / WASD: 移動 / F1: Camera切替

09 自己評価と次の改善

観点	評価	根拠 / 次の対応
技術的正確性	良	finite・range・state整合性を検証。自動テストは不足
可読性	概ね良	farm配下は責務別。GamePlaySceneは依然巨大
保守性	良	UIと状態変更を分離、constexpr利用。調整数値のJSON化余地あり
拡張性	概ね良	System追加は容易。CropTypeと表示モデルのdata-driven化が必要
実行時安全性	概ね良	nullptr/OOB/finite検証あり。filesystem異常系試験が不足
DirectX 12正当性	基盤依存	農場層はGPU資源非所有。descriptor/barrier/fenceの継続監査が必要
性能	現規模で妥当	20tile全走査。大規模畑ではupdate/draw batchingを設計

最優先は、仮の所持金・ランク表示を実ゲームシステムへ接続し、収穫→報酬→次の栽培投資までのコアループを完成させることです。その後に作物アセット、効果音、エフェクトを追加します。

— End of document —